

Vibe Coding vs. The World

The Evolution of Outsourcing: From Offshore Teams to AI Generation



A New Name for an Old Habit.

Software teams have always engaged in “vibe coding”: describing a vision in natural language (the “vibe”) and waiting for implementation. Traditionally, this meant writing a brief, handing it to a human, and reviewing the output later.

The Shift: The difference is purely **latency**. **Generative AI** closes the loop from “description” to “code” in **seconds**, not weeks.

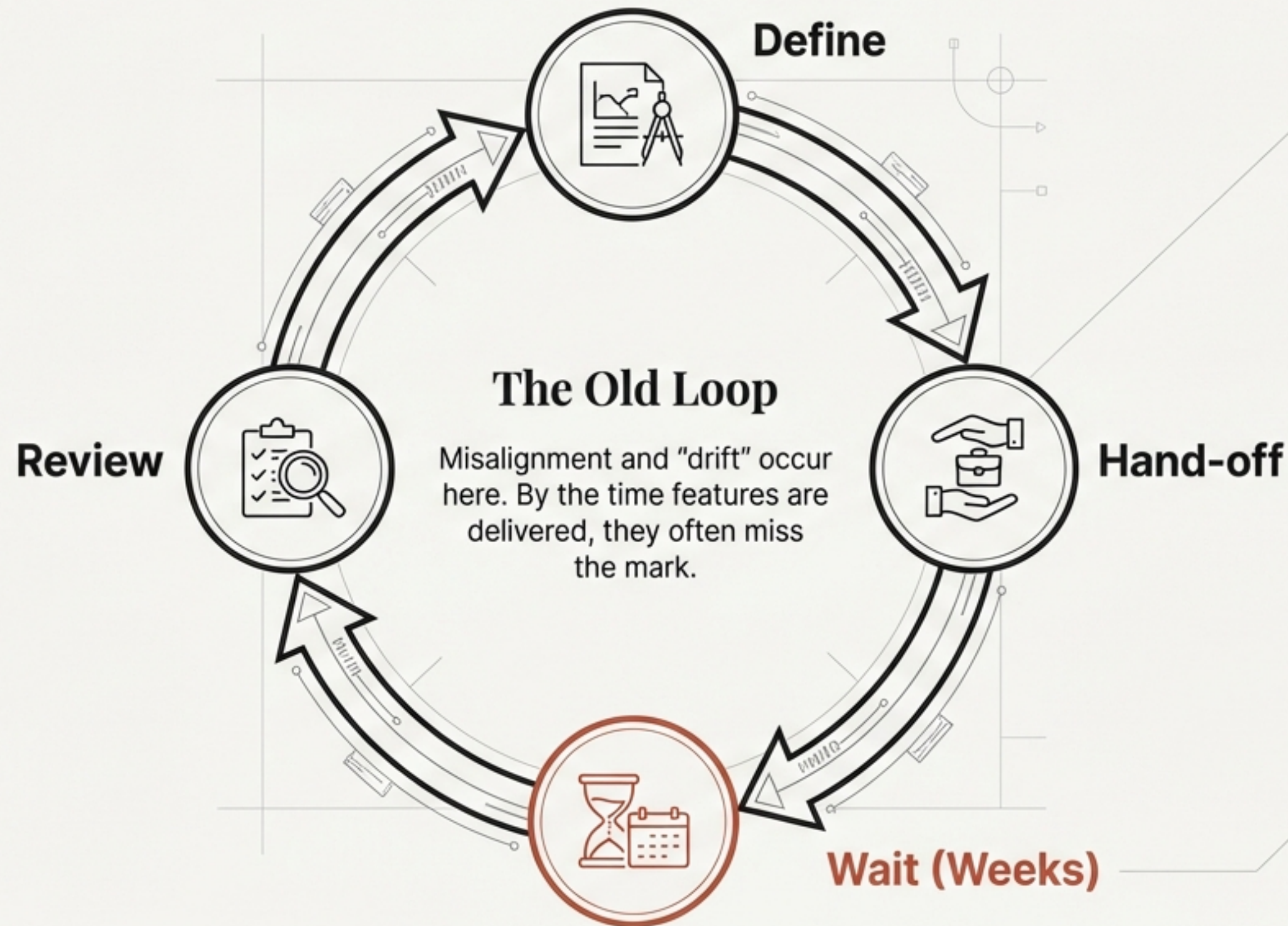
“Most software developers don’t write every line of code from scratch... they draw on existing libraries.”

— WIRED (2025)

Natural Language + Human Execution = Traditional Outsourcing

Natural Language + AI Execution = Vibe Coding

Traditional Outsourcing: The Slow Loop.



Key Data

57%

of IT projects fail due to communication issues.

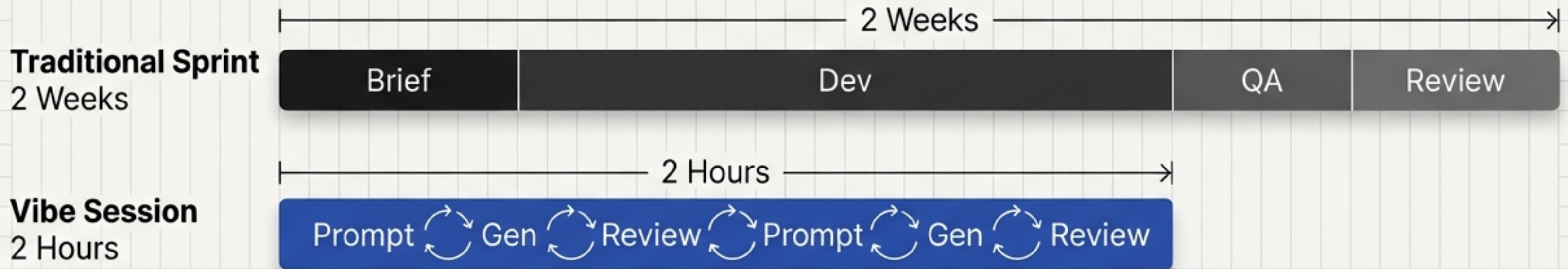
62%

of outsourced IT projects run over budget.

70%

longer timelines caused by miscommunication lags.

Compressing the Feedback Loop



Real-time Steering

Unlike offshore teams, AI is an 'always-on' pair programmer. Business stakeholders can tweak the 'vibe' and see changes immediately.

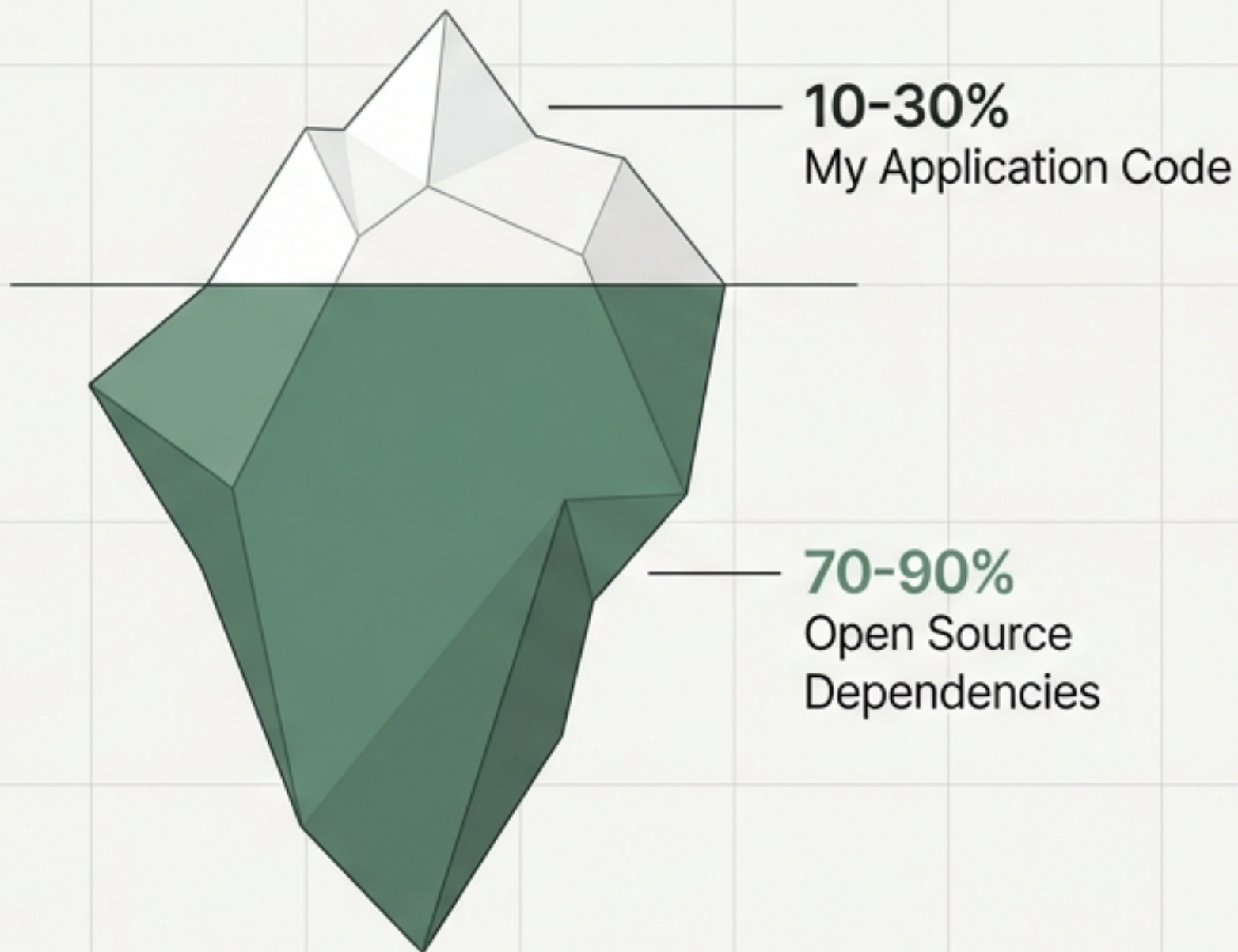
Democratisation

Non-technical stakeholders are no longer locked out. They can say 'I don't like how this behaves' and watch the adjustment happen—Agile development on steroids.

Shift: From handing off a spec into a black box → Actively steering the outcome

Libraries: The Outsourcing We Forgot We Were Doing

Vibe coding isn't the first time we stopped writing code from scratch.



The Hidden Cost: We gained productivity, but we imported the Dependency Supply Chain.

Data Point: 70–90% of modern code comes from open-source components, not in-house writing.

The Cost of Borrowed Code.

Risk 1: Bloat



Importing a mega-library for a single function. Small random projects can have over 1,000 dependencies once transitive packages are included.

Risk 2: Vulnerability



86% of codebases contain at least one known open-source vulnerability. The average application has ~911 dependencies, and 90% are outdated by over 4 years.

Case Study: Log4j



An obscure feature (JNDI lookups) that no one asked for created a global security flaw. When you use a library, you import features you didn't ask for.

Bespoke vs. Off-the-Shelf

Libraries (Open Source)



Generic, heavy, includes unused features (bloat). You import features you didn't ask for.

Vibe Coding (AI)



Specific, lean, generates *only* what you asked for. Reduces technical debt by generating exact-fit solutions.

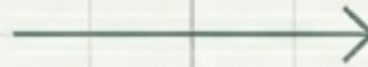
Instruction to AI: "Generate a logging module that does X and Y, and nothing fancy beyond that."

From Blind Trust to Active Inspection.

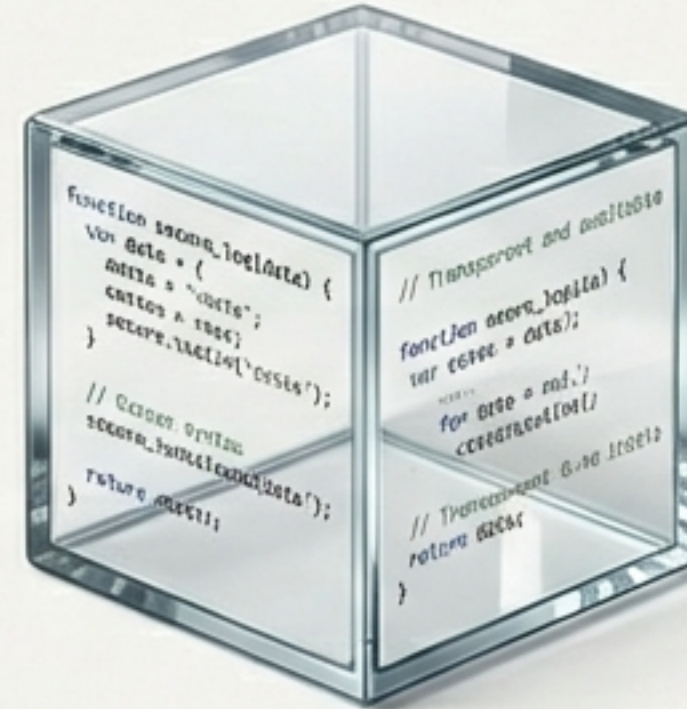
Libraries



Trust the maintainer. Vulnerabilities found *after* the fact. Opaque binary.



Vibe Coding



'Shift Left' Security. You own the code immediately. Transparent.

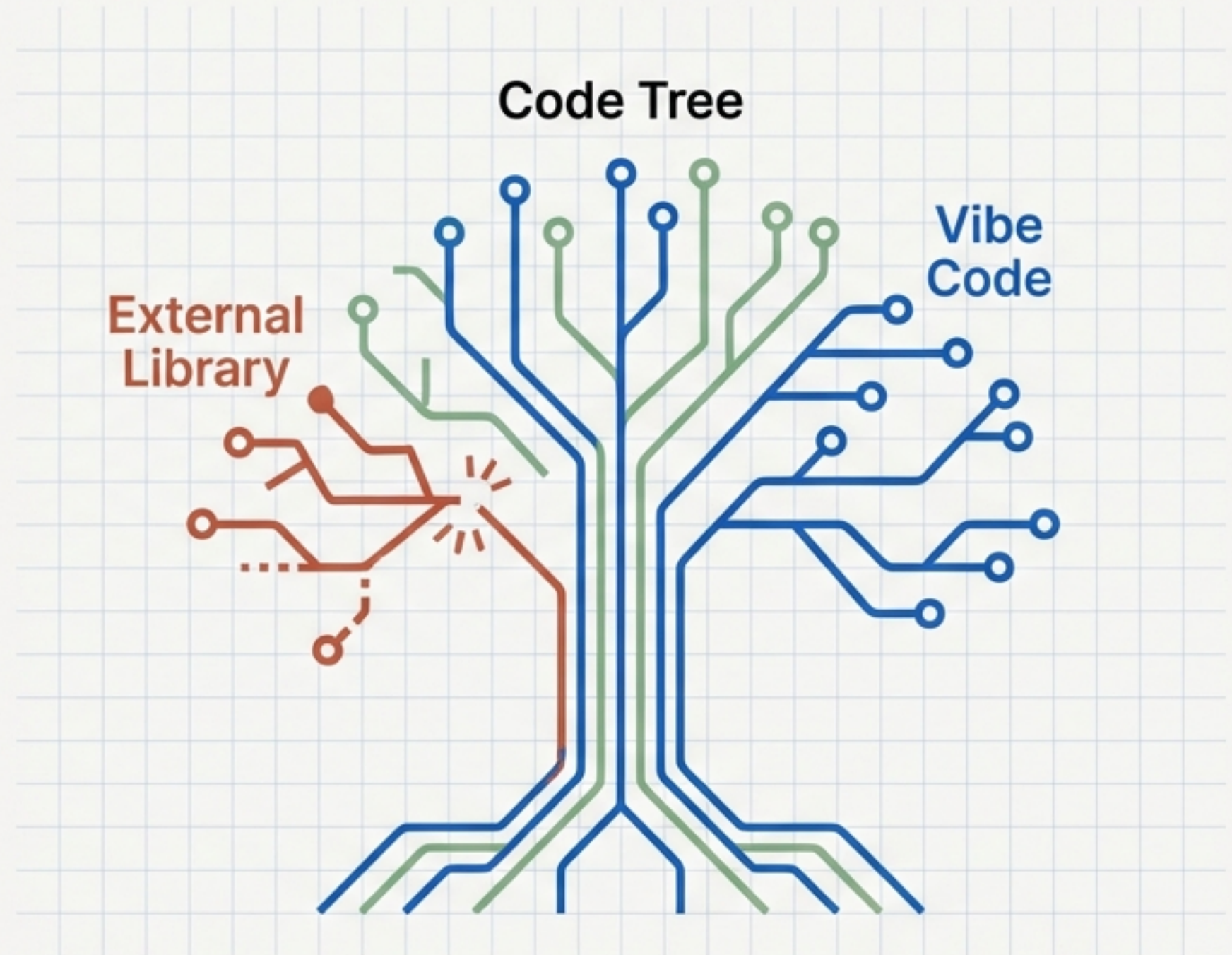
Crucial Nuance: AI code is not inherently secure—it can hallucinate packages. But because it is transparent, you can scan, review, and fix it instantly, unlike a compiled third-party package.

Taking Back the Steering Wheel

Problem with Libraries: You are at the mercy of the maintainer's roadmap. Abandoned projects become your problem.

Vibe Advantage: You fork at inception. The code follows *your* architecture and naming conventions.

Result: Coherence. The code looks like it belongs in your system, making it easier for internal teams to own and refactor.



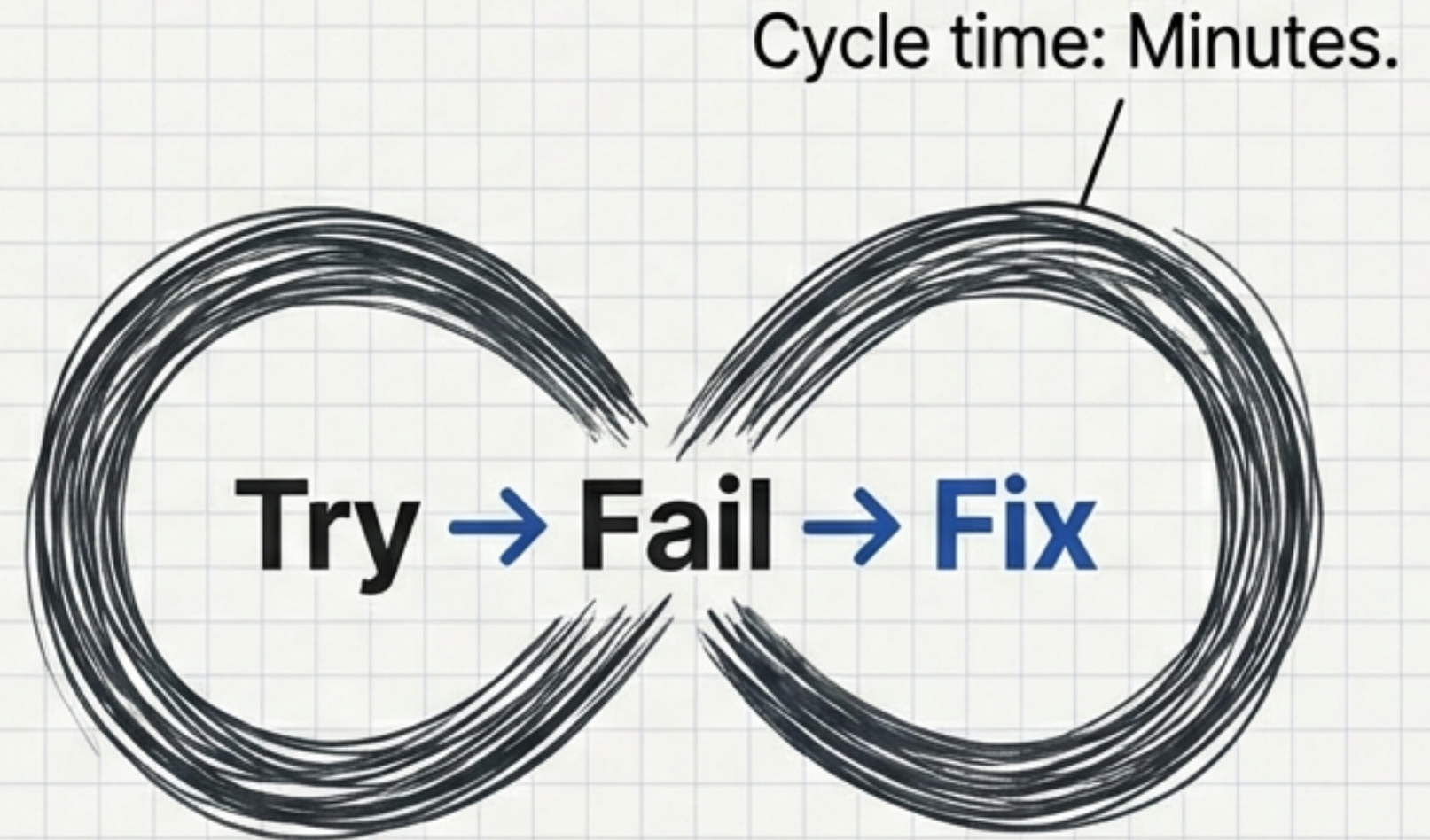
Visualizing Code Coherence and Ownership

The ‘Trial and Error’ Economy.

The cost of experimentation has collapsed.

Scenario: If a library is wrong, ripping it out is an expensive refactoring project. If AI code is wrong, you simply re-prompt in 30 seconds.

Quote: “Agile development on steroids”—a super-speed pair programmer who can rewrite the module instantly.



The AI is the Junior Dev. You are the Architect.

The Reality Check:

AI makes mistakes. It can hallucinate packages or use insecure patterns.

Only 18%

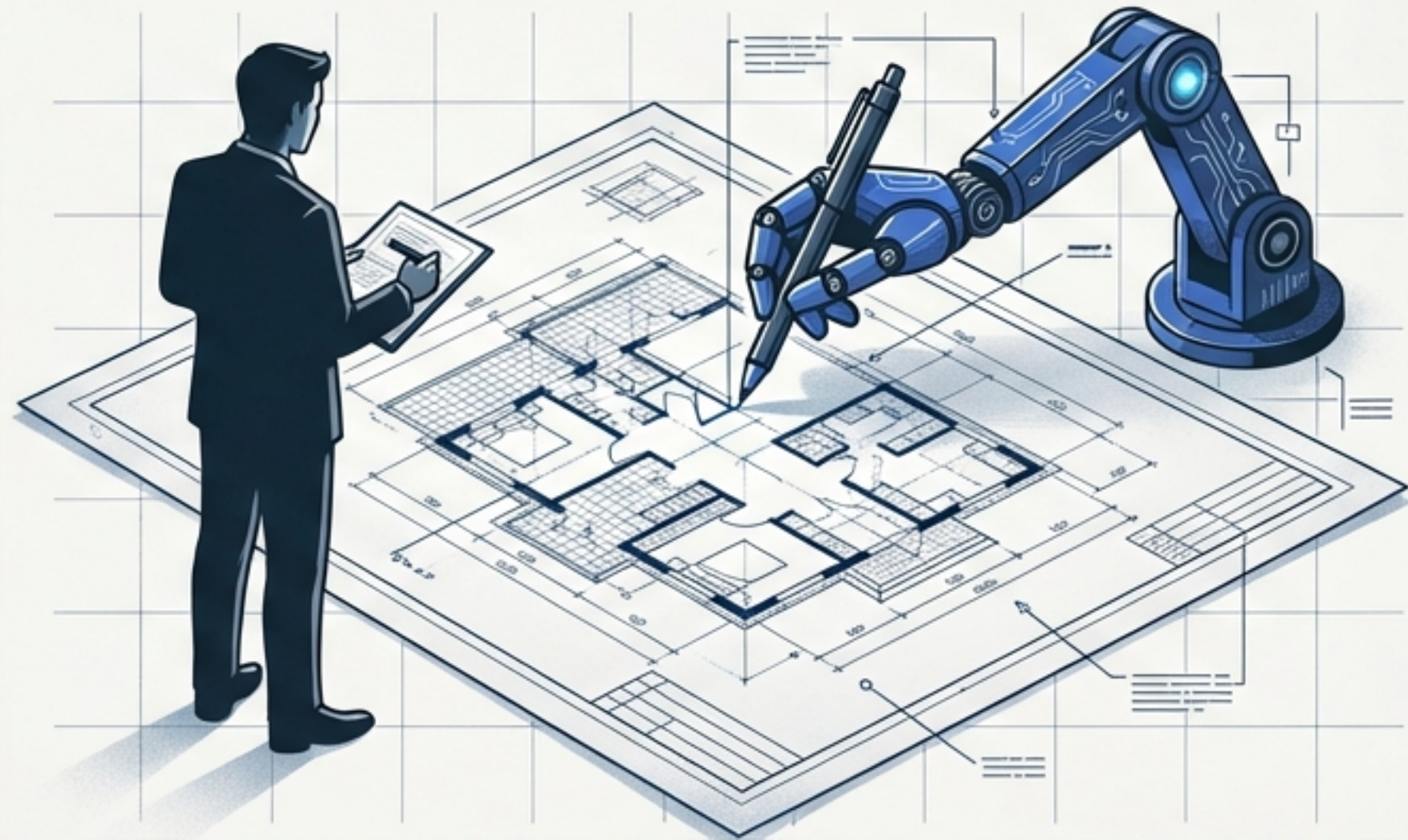
of organisations have formal policies for AI coding tools, despite high usage.

The Role Shift:

The Human in the Loop:

Shift from WRITER → REVIEWER.

Shift from TYPIST → ARCHITECT.



How to Vibe Code Safely

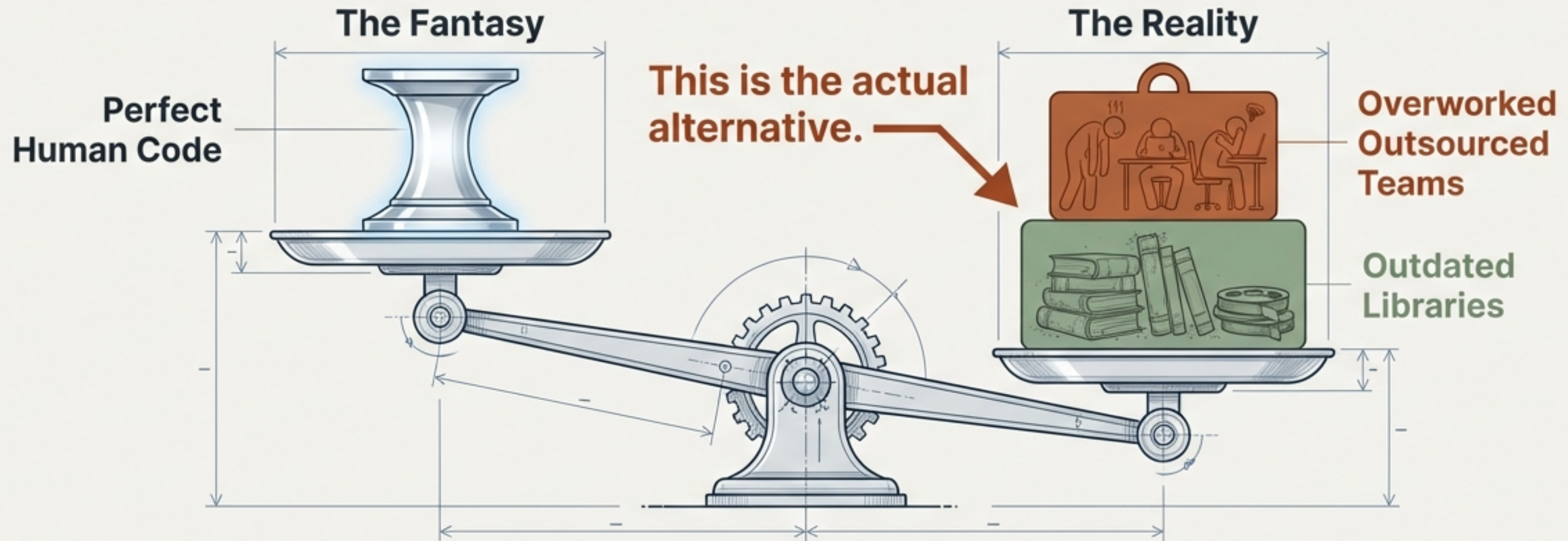
Treat AI output like a rough draft

“AI is its own worst enemy in terms of generating code that’s insecure if used naively.” — Security Researchers



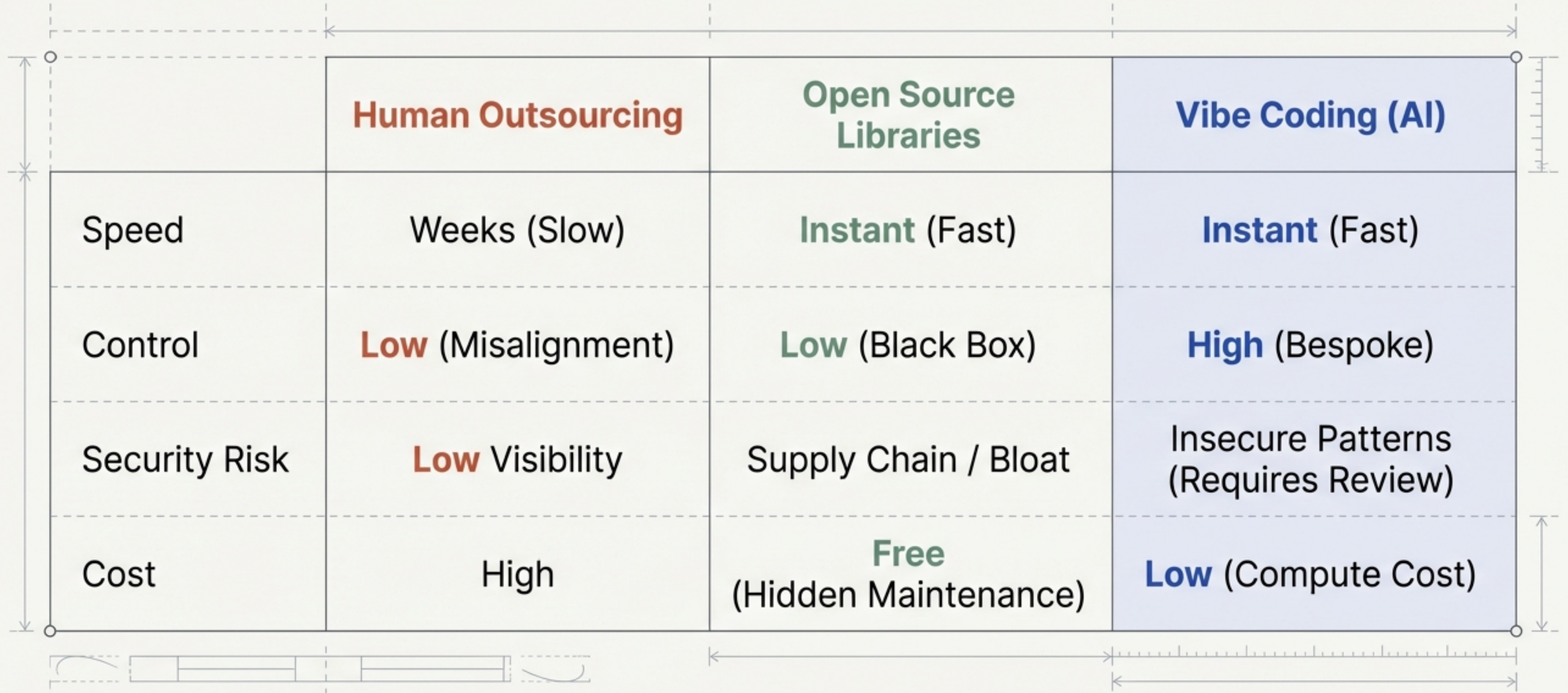
Evaluating the Alternatives Fairly.

The Fallacy: Do not compare AI code to an imaginary ideal of perfect, hand-crafted human code.



Verdict: In this light, **guided AI** often wins on speed and specificity.

The Three Eras of Code Outsourcing



	Human Outsourcing	Open Source Libraries	Vibe Coding (AI)
Speed	Weeks (Slow)	Instant (Fast)	Instant (Fast)
Control	Low (Misalignment)	Low (Black Box)	High (Bespoke)
Security Risk	Low Visibility	Supply Chain / Bloat	Insecure Patterns (Requires Review)
Cost	High	Free (Hidden Maintenance)	Low (Compute Cost)

The diagram illustrates the evolution of code outsourcing through three distinct eras, each evaluated against four key dimensions: Speed, Control, Security Risk, and Cost. The eras are Human Outsourcing, Open Source Libraries, and Vibe Coding (AI). The progression shows a shift from slow, high-cost, and low-control human outsourcing to fast, low-cost, and high-control AI-based coding, with intermediate stages in open source libraries.

Outsource the Toil, Not the Vision.

Vibe coding returns the focus to the 'Vibe'—the business requirement.

Call to Action: Embrace the tool, but keep your hands on the wheel. **Trust, but verify.**

